Practica 07

Objetivo general

Integrar el uso de comunicación serial, visualización de datos y control de periféricos digitales en un sistema embebido basado en ESP32-S3, mediante la lectura de una señal analógica, el envío y recepción de comandos por UART, la visualización de información en una pantalla OLED SSD1306 y el control de un LED RGB tipo NeoPixel.

Objetivos específicos

1. Comprender el uso básico de la comunicación UART para enviar y recibir caracteres desde el Monitor Serie de Arduino.
2. Aplicar el manejo de caracteres y texto formateado para presentar información en formato decimal, hexadecimal y de punto flotante.
3. Utilizar el convertidor analógico a digital del ESP32-S3 para leer el voltaje generado por un potenciómetro externo.
4. Implementar comunicación I2C entre el ESP32-S3 y una pantalla OLED SSD1306 para mostrar información del sistema.
5. Controlar un LED RGB tipo NeoPixel mediante una comunicación digital por un solo pin.
6. Integrar en un mismo programa la lectura analógica, la interpretación de comandos, la comunicación serial, la visualización en pantalla y el control de salidas digitales.

Resultados de aprendizaje

Al finalizar esta práctica, el estudiante estará en capacidad de:

1. Identificar diferencias generales entre tecnologías de comunicación serial como UART, I2C y comunicación digital por un solo pin.
2. Recibir caracteres desde la terminal serial e interpretarlos como comandos para controlar un sistema embebido.
3. Mostrar información numérica en diferentes formatos, incluyendo decimal, hexadecimal y voltaje con dos cifras decimales.
4. Conectar y programar una pantalla OLED SSD1306 usando comunicación I2C con el ESP32-S3.
5. Controlar el estado de un LED externo y modificar el color de un LED RGB NeoPixel integrado.
6. Explicar el funcionamiento general del circuito y del programa implementado, relacionando cada comando recibido con la acción ejecutada por el microcontrolador.

Marco teórico

Comunicación UART en el ESP32-S3

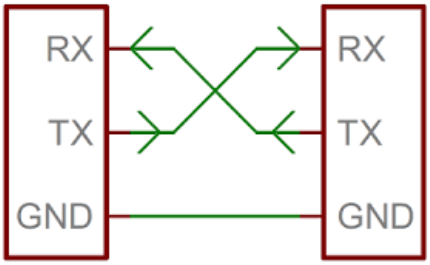
Uno de los mecanismos de comunicación más utilizados en sistemas embebidos es la comunicación serial tipo **UART**, sigla de *Universal Asynchronous Receiver-Transmitter*. Esta comunicación permite enviar y recibir información digital bit a bit entre dos dispositivos, utilizando normalmente dos líneas principales: una línea de transmisión, llamada **TX**, y una línea de recepción, llamada **RX**.

En una comunicación UART, el dispositivo que transmite envía una secuencia de bits a través de su pin TX, mientras que el dispositivo receptor lee esa información a través de su pin RX. Para que la comunicación funcione correctamente, ambos dispositivos deben compartir una referencia común de tierra (**GND**) y deben estar configurados con los mismos parámetros de comunicación, especialmente la velocidad de transmisión, conocida como **baud rate**.

Por ejemplo, cuando se configura el puerto serial a **115200 baudios**, se está indicando que la comunicación se realizará a una velocidad aproximada de 115200 símbolos por segundo. En las prácticas con Arduino y ESP32 es común utilizar instrucciones como:

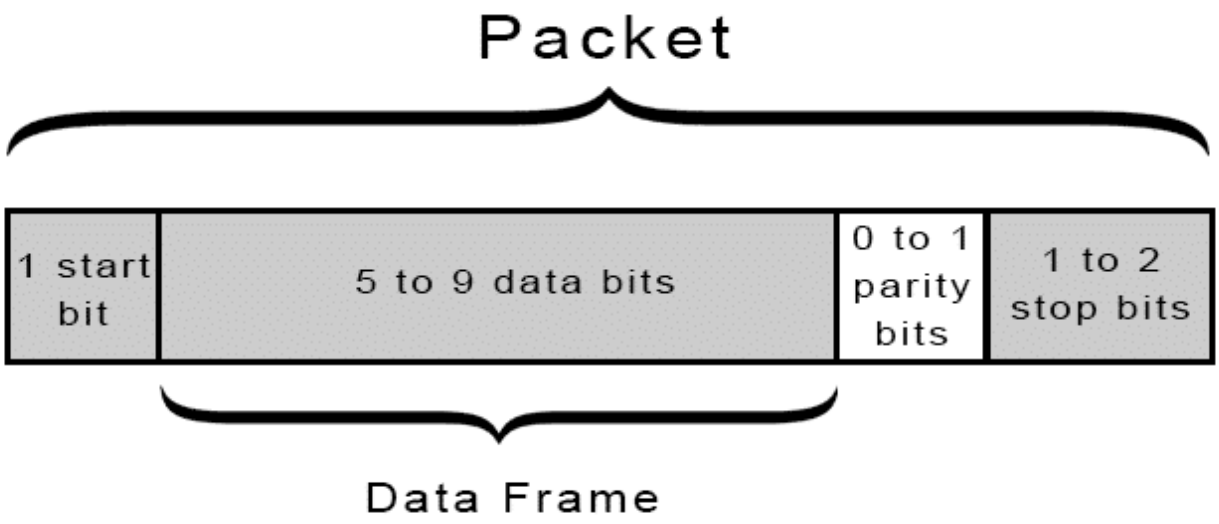
```
Serial.begin(115200);
```

Esta instrucción inicializa la comunicación serial y permite que el microcontrolador intercambie información con el computador a través del Monitor Serie del entorno de programación.



En la figura se muestra la conexión entre dos dispositivos conectados a través de un puerto uart

A diferencia de otros protocolos de comunicación, UART es una comunicación **asíncrona**. Esto significa que no utiliza una línea de reloj compartida entre los dispositivos. En lugar de ello, tanto el transmisor como el receptor deben estar previamente configurados con la misma velocidad de comunicación. Cada byte transmitido se envía acompañado de bits adicionales que permiten indicar el inicio y el final de la transmisión. De manera simplificada, una trama UART suele incluir un bit de inicio, varios bits de datos, opcionalmente un bit de paridad y uno o más bits de parada.



En la figura se muestra lo que es una trama típica en una comunicación UART, el bit de inicio siempre es un cero, después van los bits de datos que por lo general son 8 bits lo que corresponde con carácter del código ascii. Luego a un bit paridad que es opcional y se calcula mediante un algoritmo de control de comunicación. Luego los bits de parada, que comúnmente es uno, pero pueden ser hasta 2 bits.

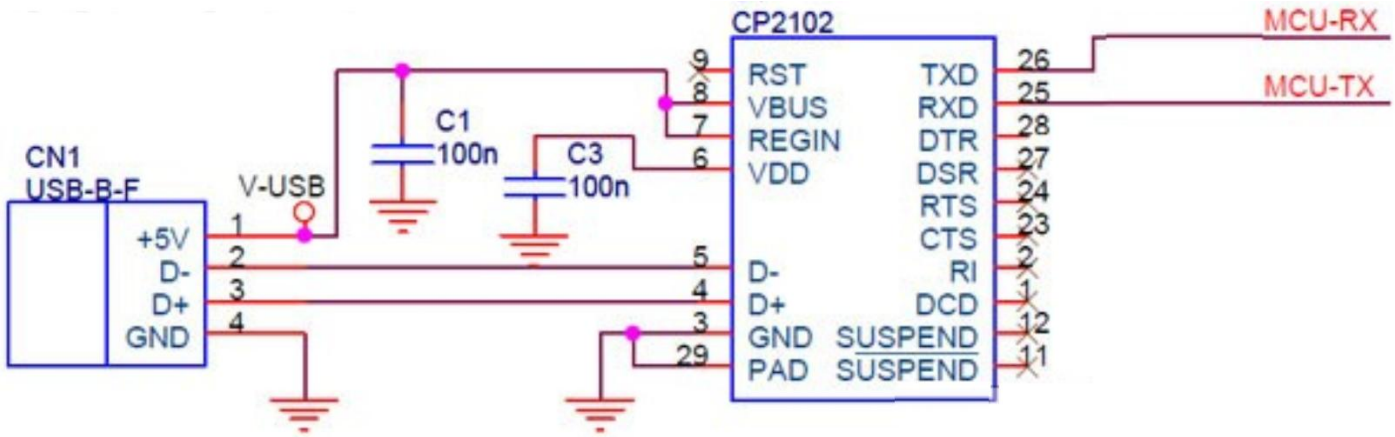
UART y USB: ¿por qué se necesita un puente?

Aunque UART es muy usado entre microcontroladores, sensores, módulos de comunicación y otros circuitos electrónicos, los computadores modernos normalmente no tienen un puerto UART TTL disponible directamente. En cambio, los computadores utilizan puertos **USB**.

Esto genera una diferencia importante: el microcontrolador trabaja internamente con señales UART de nivel lógico, usualmente de **3.3 V** en el caso del ESP32-S3, mientras que el computador se comunica mediante el protocolo USB. Por esta razón, muchas tarjetas de desarrollo incorporan un circuito integrado adicional que actúa como puente entre ambos mundos.

Ese circuito se conoce como **convertidor USB a UART** o **puente USB-Serial**. Algunos de los chips más utilizados para esta función son el **CP2102**, el **CH340**, el **CH9102**, el **FT232** y otros dispositivos similares.

La función de este chip es recibir los datos enviados por el computador a través del puerto USB y convertirlos en señales UART compatibles con el microcontrolador. En sentido contrario, también recibe los datos UART enviados por el ESP32-S3 y los convierte en datos USB que el computador puede interpretar como un puerto serial virtual.



En la figura se muestra cómo un CP 2102 toma las señales de un puerto USB (a la izquierda) y las transforma en las señales Tx y Rx, es decir las señales de transmisión y recepción del UART del microcontrolador. Note que habitualmente se abrevia la palabra microcontrolador como MCU (MicroController Unit).

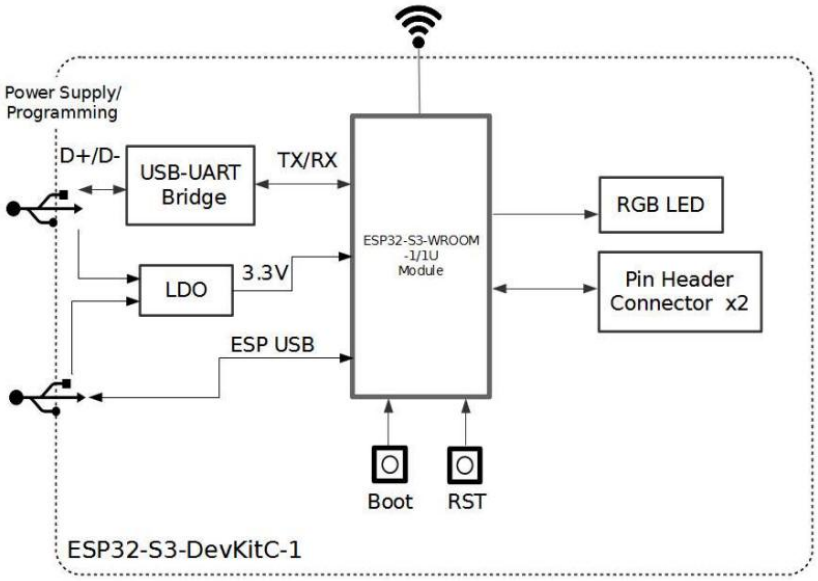
Gracias a este puente, cuando se conecta la tarjeta al computador, el sistema operativo detecta un puerto serial virtual. En Windows suele aparecer como un puerto **COM**, mientras que en Linux o macOS puede aparecer como un dispositivo tipo /dev/ttyUSBx, /dev/ttyACMx o similar. Ese puerto es el que se selecciona desde el IDE de Arduino o desde otro entorno de programación para cargar programas y abrir el Monitor Serie.

En muchas tarjetas de entrenamiento basadas en ESP32 o ESP32-S3, este puente USB-UART también permite cargar el programa al microcontrolador. Para ello, la tarjeta puede incluir circuitos adicionales que controlan automáticamente las señales de arranque y reinicio del microcontrolador, de manera que no sea necesario presionar manualmente los botones **BOOT** y **RESET** durante la programación.

El ESP32-S3 y su puerto USB nativo

Una diferencia importante del ESP32-S3 frente a varios modelos anteriores del ESP32 es que el ESP32-S3 incorpora capacidades de comunicación USB directamente en el microcontrolador. Esto significa que, dependiendo del diseño de la tarjeta de desarrollo, puede existir un puerto USB conectado directamente al ESP32-S3, sin necesidad de pasar por un chip externo USB-UART.

Este puerto USB nativo puede utilizarse para varias funciones, entre ellas programación, depuración y comunicación serial mediante USB. En algunas tarjetas, esto permite que el computador se comunique directamente con el microcontrolador usando una interfaz USB interna, lo cual puede simplificar el diseño de hardware y ofrecer posibilidades más avanzadas.



En el diagrama de bloques del módulo de desarrollo ESP32-S3-DevKit1 se observa que el sistema ofrece dos alternativas de conexión USB hacia un computador: la primera, ubicada en la parte superior izquierda del esquema, emplea un convertidor USB a UART, el cual permite que el computador se comunique con el microcontrolador a través de una interfaz serial; la segunda corresponde a la conexión directa al puerto USB nativo integrado en el ESP32-S3, una de las ventajas de esta familia frente a generaciones anteriores. En ambos casos, del puerto USB también se obtiene la alimentación de 5 V suministrada por el computador, la cual entra a un regulador de voltaje tipo LDO (Low Dropout, es decir, regulador de baja caída), encargado de reducir y estabilizar ese nivel hasta 3.3 V, tensión con la que se alimenta el microcontrolador y la lógica principal de la tarjeta.

El USB nativo del ESP32-S3 representa una ventaja frente a tarjetas basadas en algunos modelos anteriores del ESP32, donde normalmente era obligatorio usar un chip externo como CP2102 o CH340 para programar la tarjeta y comunicarse con el computador. En el ESP32-S3, dependiendo de la tarjeta, puede haber dos caminos posibles: uno mediante el puente USB-UART tradicional y otro mediante el puerto USB propio del microcontrolador.

En esta práctica se utilizará principalmente la comunicación serial desde el punto de vista del usuario, es decir, el envío y recepción de información mediante el Monitor Serie. Más adelante se podrá estudiar con mayor detalle el puerto USB nativo del ESP32-S3, sus modos de funcionamiento, sus ventajas para depuración y sus diferencias con el uso clásico de un puente USB-UART.

Transmisión de caracteres usando UART y códigos ASCII

En la comunicación serial por UART, la información se transmite como una secuencia de bits. Sin embargo, desde el punto de vista del programador, muchas veces esos bits se interpretan como **caracteres de texto**. Esto permite enviar letras, números, símbolos y comandos escritos desde el computador hacia el microcontrolador.

Una de las codificaciones más conocidas para representar caracteres es la tabla **ASCII**, sigla de *American Standard Code for Information Interchange*. En su versión clásica, ASCII utiliza **7 bits** para representar 128

caracteres diferentes. Dentro de estos caracteres se incluyen letras mayúsculas, letras minúsculas, números, signos de puntuación y algunos caracteres de control.

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

La tabla de códigos ASCII permite relacionar cada carácter con un valor numérico específico. En su versión clásica de 7 bits, ASCII representa valores entre 0 y 127. Los códigos del 0 al 31 corresponden, en general, a caracteres de control, los cuales no se imprimen directamente en pantalla, sino que cumplen funciones especiales, como salto de línea, retorno de carro, tabulación o fin de transmisión. A partir del código 32 se encuentran los caracteres imprimibles, entre ellos espacios, números, signos de puntuación, letras mayúsculas y letras minúsculas. En la tabla se muestran usualmente los valores equivalentes de cada carácter tanto en formato decimal como hexadecimal, lo cual resulta útil porque en programación de sistemas embebidos es común trabajar con ambas representaciones.

Por ejemplo, en ASCII la letra mayúscula **A** tiene el valor decimal **65**, mientras que la letra minúscula **a** tiene el valor decimal **97**. La diferencia entre ambas es:

$$97 - 65 = 32$$

Esto mismo ocurre con las demás letras del alfabeto inglés. Por ejemplo:

'B' = 66 'b' = 98
'C' = 67 'c' = 99
'Z' = 90 'z' = 122

En todos estos casos, para pasar de una letra mayúscula a su equivalente en minúscula basta con sumar **32** al código ASCII de la letra mayúscula.

Aunque UART puede utilizarse para transmitir información binaria de muchas formas diferentes, en esta práctica se usará principalmente para transmitir **caracteres y cadenas de texto**. Esto resulta conveniente porque permite interactuar fácilmente con el ESP32-S3 desde el Monitor Serie del entorno de Arduino.

Ejemplo: convertir una letra mayúscula en minúscula

El siguiente programa recibe un carácter desde el Monitor Serie. Luego, al valor ASCII recibido se le suma una constante igual a **32**, y finalmente se transmite de regreso el carácter resultante.

La idea del ejemplo es mostrar que los caracteres también pueden interpretarse como números. Por eso, si se recibe el carácter 'A', internamente el microcontrolador puede interpretarlo como el número **65**. Al sumarle **32**, se obtiene **97**, que corresponde al carácter 'a'.

```
1
2  // Práctica 07 - Ejemplo básico de recepción de caracteres por UART
3  // Conversión de una letra mayúscula a minúscula usando códigos ASCII
4
5  const int DIFERENCIA_MAYUSCULA_MINUSCULA = 32;
6
7  void setup() {
8    Serial.begin(115200);
9    delay(1000);
10   Serial.println("Ejemplo de conversion ASCII por UART");
11   Serial.println("Envia una letra MAYUSCULA desde el Monitor Serie.");
12   Serial.println("El sistema sumara 32 al codigo ASCII recibido.");
13   Serial.println();
14 }
15
16 void loop() {
17   if (Serial.available() > 0) {
18     char caracterRecibido = Serial.read();
19     char caracterConvertido = caracterRecibido + DIFERENCIA_MAYUSCULA_MINUSCULA;
20
21     Serial.print("Caracter recibido: ");
22     Serial.println(caracterRecibido);
23
24     Serial.print("Codigo ASCII recibido: ");
25     Serial.println((int)caracterRecibido);
26
27     Serial.print("Caracter convertido: ");
28     Serial.println(caracterConvertido);
29
30     Serial.print("Codigo ASCII convertido: ");
31     Serial.println((int)caracterConvertido);
32
33     Serial.println("----------------------------");
34   }
35 }
```

Para probar este programa, el estudiante debe cargarlo en el ESP32-S3 y abrir el Monitor Serie a **115200 baudios**. Luego debe enviar letras mayúsculas individuales, por ejemplo:

A
B
C
Z

Si se envía la letra **A**, el sistema deberá responder mostrando que recibió el carácter **A**, cuyo código ASCII es **65**, y que al sumarle **32** se obtiene el carácter **a**, cuyo código ASCII es **97**.

Aspecto importante del programa

El programa anterior funciona correctamente si el usuario envía letras mayúsculas entre **A** y **Z**. Sin embargo, el programa **no verifica** si el carácter recibido realmente corresponde a una letra mayúscula.

Esto significa que si el usuario envía un número, un símbolo, una letra minúscula o incluso un carácter de control, el programa también le sumará **32**. En ese caso, el resultado puede no tener sentido desde el punto de vista del usuario.

Por ejemplo, si se envía el carácter '1', cuyo código ASCII es **49**, el programa sumará 32 y obtendrá **81**, que corresponde al carácter 'Q'. Es decir, el programa no está "convirtiendo números", sino simplemente sumando un valor fijo al código ASCII recibido.

También se debe tener cuidado con la configuración del Monitor Serie. Si está configurado para enviar "Nueva línea" o "Retorno de carro", al presionar Enter se pueden enviar caracteres adicionales como \n o \r. Estos caracteres también serían procesados por el programa.

Por esta razón, en un programa más robusto se debe verificar primero si el carácter recibido pertenece al rango esperado.

Tarea para los estudiantes

Modifique el programa anterior para que solo convierta caracteres que correspondan a letras mayúsculas entre **A** y **Z**.

El programa debe cumplir lo siguiente:

1. Leer un carácter desde el Monitor Serie.
2. Verificar si el carácter recibido está entre 'A' y 'Z'.
3. Si el carácter es una letra mayúscula válida, convertirlo a minúscula sumando 32.
4. Si el carácter no es una letra mayúscula, mostrar un mensaje indicando que el carácter recibido no es válido para esta conversión.

Una posible condición para validar el carácter recibido es:

```
if (caracterRecibido >= 'A' && caracterRecibido <= 'Z') {
  // El carácter es una letra mayúscula válida
}
```

Esta validación es importante porque en sistemas embebidos reales no se debe asumir que el usuario, el computador o el dispositivo conectado siempre enviará datos correctos. Un sistema bien diseñado debe verificar la información recibida antes de procesarla.

Texto con formato usando Serial.printf()

En programación de sistemas embebidos es muy común necesitar imprimir información de manera organizada. Por ejemplo, no basta con mostrar únicamente un número en la terminal serial; muchas veces se requiere acompañarlo de un texto explicativo, mostrarlo con cierta cantidad de decimales, presentarlo en decimal, en hexadecimal o combinar varios datos en una misma línea.

A esto se le conoce como **texto con formato**. El texto con formato permite construir mensajes en los que se mezclan cadenas de texto fijas con valores de variables. Para ello se utilizan funciones tipo printf, ampliamente usadas en el lenguaje C y también disponibles en el entorno Arduino para el ESP32.

En el caso del ESP32-S3, se puede utilizar directamente la función **Serial.printf();**

Esta función permite enviar texto formateado al Monitor Serie. Dentro del texto se colocan unos símbolos especiales llamados especificadores de formato, que indican cómo debe imprimirse cada dato.

Algunos especificadores comunes son:

- ***%d*** *Imprime un número entero en formato decimal*
- ***%X*** *Imprime un número entero en formato hexadecimal con letras mayúsculas*
- ***%x*** *Imprime un número entero en formato hexadecimal con letras minúsculas*
- ***%f*** *Imprime un número de punto flotante*
- ***%.2f*** *Imprime un número de punto flotante con dos cifras decimales*

Por ejemplo:

```
int valor = 255;
float voltaje = 3.1416;

Serial.printf("Valor decimal: %d\n", valor);
Serial.printf("Valor hexadecimal: %X\n", valor);
Serial.printf("Voltaje: %.2f V\n", voltaje);
```

En este ejemplo, el mismo valor entero puede imprimirse en decimal o en hexadecimal. Además, el valor de voltaje puede imprimirse como número de punto flotante con dos cifras decimales.

El símbolo \n indica un salto de línea, de manera que cada mensaje aparece en una línea diferente del Monitor Serie.

Ejercicio: lectura de un potenciómetro e impresión con formato

En este ejercicio se conectará un potenciómetro al ESP32-S3 para generar una señal analógica variable entre **0 V y 3.3 V**. Esta señal será leída mediante el convertidor analógico a digital del microcontrolador, conocido como **ADC**.

El potenciómetro debe conectarse de la siguiente manera:

- Un extremo del potenciómetro a **3.3 V**.
- El otro extremo del potenciómetro a **GND**.
- El pin central del potenciómetro al pin analógico seleccionado del ESP32-S3.

Al girar el eje del potenciómetro, el voltaje en el terminal central cambia gradualmente entre 0 V y 3.3 V. El ADC del ESP32-S3 convierte ese voltaje en un número entero. Si se utiliza una resolución de 12 bits, el valor leído estará típicamente entre **0 y 4095**.

Ese valor leído desde el ADC se imprimirá en el Monitor Serie de tres formas diferentes:

1. Como número entero en formato decimal.
2. Como número entero en formato hexadecimal.
3. Como voltaje calculado en formato de punto flotante con dos cifras decimales.

Para convertir el valor leído por el ADC en voltaje, se puede utilizar la siguiente ecuación:

$$\text{voltaje} = \text{valorADC} \times 3.3 / 4095.0$$

Donde:

valorADC = número leído por el convertidor analógico a digital
3.3 = voltaje máximo esperado en la entrada analógica
4095.0 = valor máximo para una conversión ADC de 12 bits

En este programa, la variable valorADC almacena el número entero entregado por el convertidor analógico a digital. Luego, ese valor se convierte en voltaje mediante una ecuación sencilla basada en una regla de tres.

```ino
const int pinPotenciometro = 5;      // Pin conectado al terminal central del potenciómetro
const float voltajeReferencia = 3.3; // Voltaje máximo esperado en la entrada ADC
const int resolucionADC = 4095;      // Valor máximo para ADC de 12 bits

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("Lectura de potenciometro con texto formateado");
  Serial.println("Valor ADC en decimal, hexadecimal y voltaje calculado");
  Serial.println();

  // Configuración opcional para ESP32
  analogReadResolution(12);
}

void loop() {
  int valorADC = analogRead(pinPotenciometro);

  float voltaje = (valorADC * voltajeReferencia) / resolucionADC;

  Serial.printf("ADC decimal: %d | ADC hexadecimal: 0x%03X | Voltaje: %.2f V\n",
                valorADC,
                valorADC,
                voltaje);

  delay(500);
}
```

La instrucción:

```ino
Serial.printf("ADC decimal: %d | ADC hexadecimal: 0x%03X | Voltaje: %.2f V\n",
              valorADC,
              valorADC,
              voltaje);
```

En esta instrucción se imprimen tres representaciones diferentes del mismo proceso de medición. El primer %d imprime el valor ADC como número decimal. El segundo especificador, 0x%03X, imprime el mismo valor en hexadecimal, agregando el prefijo 0x. Finalmente, %.2f imprime el voltaje calculado como número de punto flotante con dos cifras decimales.

En el monitor serial se vería: por ejemplo:

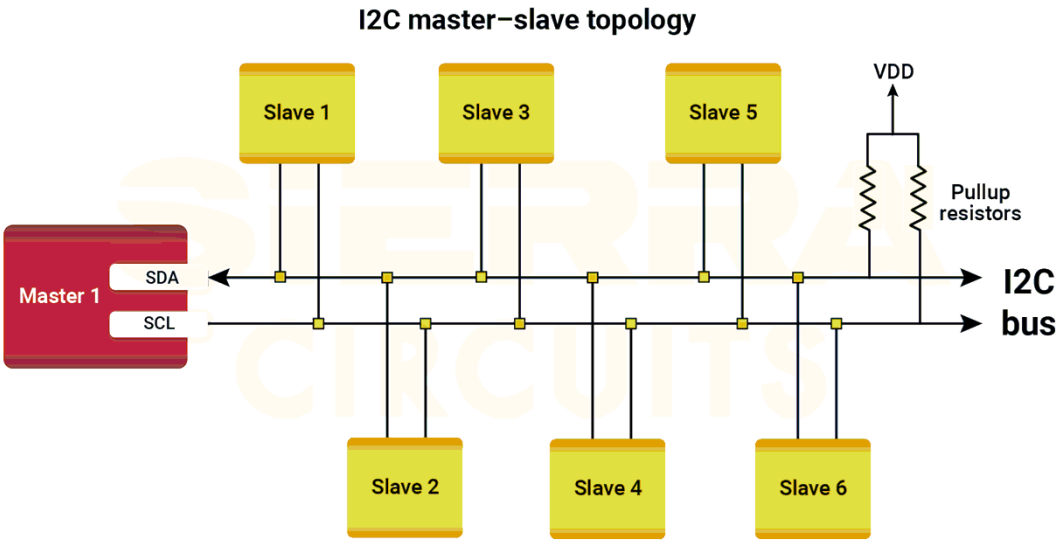
```
ADC decimal: 2048 | ADC hexadecimal: 0x800 | Voltaje: 1.65 V
ADC decimal: 3072 | ADC hexadecimal: 0xC00 | Voltaje: 2.48 V
ADC decimal: 4095 | ADC hexadecimal: 0xFFF | Voltaje: 3.30 V
```

Comunicación I2C y pantalla OLED SSD1306

Otra tecnología de comunicación serial muy utilizada en sistemas embebidos es **I2C**, sigla de *Inter-Integrated Circuit*. En esta práctica no se estudiará en detalle cómo se forman las tramas internas de comunicación ni cómo se transmiten bit a bit los datos en el bus I2C. Por ahora, lo más importante es comprender su utilidad general y cómo se puede emplear para conectar periféricos al ESP32-S3.

A diferencia de UART, que normalmente se usa como una comunicación **punto a punto**, es decir, entre un dispositivo transmisor y un dispositivo receptor, I2C permite conectar **múltiples dispositivos sobre el mismo bus de comunicación**. Este esquema se suele describir como una comunicación **maestro-esclavo**, donde el maestro generalmente es el microcontrolador y los esclavos son los dispositivos periféricos conectados a él.

En este caso, el maestro será el **ESP32-S3** y el periférico conectado será una pantalla OLED basada en el controlador **SSD1306**. Sin embargo, en un sistema embebido real podrían conectarse otros dispositivos I2C, como sensores de temperatura, sensores de presión, acelerómetros, conversores analógico-digitales externos, expansores de puertos, memorias, relojes de tiempo real y otros periféricos usados en sistemas electrónicos y de bioinstrumentación.



La comunicación I2C utiliza principalmente dos líneas:

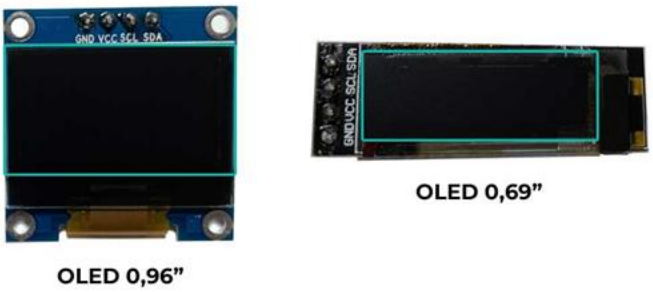
SDA: línea de datos
SCL: línea de reloj

La línea **SDA** transporta los datos, mientras que la línea **SCL** transporta la señal de reloj que sincroniza la comunicación. Además de estas dos líneas, todos los dispositivos conectados deben compartir alimentación y tierra, según los requisitos de cada módulo.

Pantalla OLED SSD1306

La pantalla **SSD1306** es una pantalla OLED pequeña, económica y muy común en proyectos con Arduino, ESP32 y otros microcontroladores. Estas pantallas suelen encontrarse en resoluciones como **128 × 32 píxeles** o **128 × 64 píxeles**.

Esto significa que la pantalla está formada por una matriz de puntos o píxeles. En una pantalla de **128 × 32**, hay 128 píxeles horizontales y 32 píxeles verticales. En una pantalla de **128 × 64**, hay 128 píxeles horizontales y 64 píxeles verticales. Aunque su tamaño físico es pequeño, estas pantallas permiten mostrar texto, números, símbolos simples y gráficos básicos.



Pantallas SSD1306 de 128x64 y 128x64 pixeles

Este tipo de visualización es muy útil porque permite que un sistema embebido muestre información sin depender siempre del computador o del Monitor Serie. En aplicaciones de bioinstrumentación, una pantalla de este tipo podría utilizarse para mostrar lecturas de sensores, estados del sistema, mensajes de advertencia, parámetros de configuración o resultados básicos de una medición.

Conexión básica de la pantalla SSD1306 al ESP32-S3

Las pantallas SSD1306 con interfaz I2C normalmente tienen cuatro pines principales:

VCC: alimentación del módulo
GND: tierra
SCL: línea de reloj I2C
SDA: línea de datos I2C

Para la tarjeta ESP32-S3 utilizada en esta práctica, se propone usar los pines I2C que se han venido trabajando en clase:

SDA → GPIO 8
SCL → GPIO 9

La conexión sería entonces:

SSD1306 VCC → 3.3 V del ESP32-S3
SSD1306 GND → GND del ESP32-S3
SSD1306 SDA → GPIO 8 del ESP32-S3
SSD1306 SCL → GPIO 9 del ESP32-S3

Es importante respetar la conexión entre **SDA** y **SCL**, ya que cada línea cumple una función diferente. Si se invierten estos pines, la comunicación no funcionará correctamente y el programa no podrá enviar información a la pantalla. También debe verificarse que la pantalla trabaje con niveles compatibles con **3.3 V**, ya que el ESP32-S3 utiliza lógica de 3.3 V.

Aunque algunos módulos SSD1306 aceptan alimentación de 5 V porque incluyen regulador o adaptación interna, en esta práctica se recomienda alimentar la pantalla desde **3.3 V** para mantener compatibilidad eléctrica con el ESP32-S3 y reducir el riesgo de daño en los pines del microcontrolador.

Para poder comunicarse con la pantalla desde Arduino, normalmente se utilizan bibliotecas como:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

La biblioteca Wire.h permite manejar la comunicación I2C. Las bibliotecas Adafruit_GFX.h y Adafruit_SSD1306.h facilitan el dibujo de texto, números y gráficos básicos en la pantalla.

En esta práctica, la pantalla SSD1306 se utilizará como una salida visual complementaria al Monitor Serie. Es decir, la información podrá enviarse tanto al computador por UART como a la pantalla por I2C, permitiendo comparar dos formas diferentes de presentar datos en un sistema embebido.

Ejemplo:

```
// Configuración de la pantalla OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
// Variables de ejemplo
int valorEntero = 2048;
float valorFlotante = 1.65;

void setup() {
  Serial.begin(115200);
  // Inicializa el bus I2C en los pines indicados
  Wire.begin(SDA_PIN, SCL_PIN);
  // Inicializa la pantalla SSD1306
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("No se encontro la pantalla SSD1306");
    while (true);
  }
  display.clearDisplay();
  // Primera línea: número entero en tamaño 1
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print("Entero: ");
  display.print(valorEntero);
  // Segunda línea: número flotante en tamaño 2
  display.setTextSize(2);
  display.setCursor(0, 14);
  display.print(valorFlotante, 2);

  // Actualiza la pantalla
  display.display();
}
void loop() {
  // En este ejemplo no se actualiza nada continuamente
}
```

El programa inicializa la comunicación serial del ESP32-S3 y configura el bus I2C usando los pines **GPIO 8** para SDA y **GPIO 9** para SCL, con el fin de comunicarse con una pantalla OLED **SSD1306**. Luego verifica que la pantalla haya sido detectada correctamente en la dirección I2C indicada, borra cualquier contenido previo, selecciona el color del texto y muestra dos datos de ejemplo: en la primera línea imprime un número entero en tamaño de texto 1 y en la segunda línea imprime un número de punto flotante con dos decimales en tamaño de texto 2. Los estudiantes deben estudiar cuidadosamente qué hace cada instrucción del programa, especialmente las funciones relacionadas con Wire, display.begin(), display.clearDisplay(), display.setTextSize(), display.setCursor(), display.print() y display.display(). Después de revisar el código, deben cargarlo en el ESP32-S3, conectar correctamente la pantalla SSD1306 y verificar su funcionamiento en el montaje físico.

Comunicación por un solo pin usando el LED RGB NeoPixel

Además de tecnologías de comunicación serial como **UART** e **I2C**, existen otros esquemas de comunicación más simples que permiten controlar dispositivos sencillos utilizando una sola línea de datos. Este es el caso de los LEDs RGB inteligentes tipo **NeoPixel**, también conocidos comúnmente como LEDs **WS2812** o compatibles.

A diferencia de un LED RGB convencional, que normalmente requiere una conexión independiente para controlar cada color, un LED NeoPixel integra en un solo encapsulado tres LEDs internos: uno rojo, uno verde y uno azul. Sin embargo, lo más importante es que también incorpora un **controlador digital interno**. Este controlador recibe información desde el microcontrolador y se encarga de ajustar la intensidad de cada uno de los tres colores.

Gracias a este controlador digital, el microcontrolador no necesita generar directamente tres señales PWM independientes para controlar el rojo, el verde y el azul. En lugar de eso, el ESP32-S3 envía una secuencia de datos por un solo pin. Esa información le indica al NeoPixel qué intensidad debe tener cada componente de color. Por ejemplo, se le puede ordenar que encienda rojo al máximo, verde apagado y azul apagado; o también que combine diferentes intensidades para producir colores como amarillo, cian, magenta, blanco o tonos intermedios.

En la tarjeta de desarrollo ESP32-S3 utilizada en esta práctica, el LED RGB tipo NeoPixel ya viene conectado internamente a un pin específico del microcontrolador. En muchas tarjetas ESP32-S3, este LED está conectado al **GPIO 48**, aunque los estudiantes deben verificar la serigrafía o la documentación específica de la tarjeta que están usando. Esta conexión interna permite controlar el LED RGB sin necesidad de armar un circuito externo adicional.



Es importante tener presente que ese pin ya está conectado físicamente al LED NeoPixel de la tarjeta. Por esta razón, se debe tener cuidado si se desea usar ese mismo GPIO para conectar otros dispositivos externos. Si otro circuito se conecta al mismo pin, pueden aparecer comportamientos extraños, conflictos eléctricos o fallos en el control del LED. En general, si un pin ya está asociado a un periférico integrado de la tarjeta, como un LED RGB, debe verificarse antes de usarlo para otra función.

En esta práctica, el NeoPixel se utilizará como una salida visual controlada por software. Más adelante se enviarán comandos de texto desde el Monitor Serie, y el ESP32-S3 interpretará esos comandos para cambiar el color del LED RGB. De esta manera, los estudiantes podrán relacionar tres conceptos importantes: recepción de texto por UART, interpretación de comandos y control de un periférico digital mediante una línea de datos.

Ejemplo:

```
#include <Adafruit_NeoPixel.h>
// Pin donde está conectado el NeoPixel integrado
#define NEOPIXEL_PIN 48
// Número de LEDs NeoPixel conectados
#define NUM_PIXELS 1
// Creación del objeto NeoPixel
Adafruit_NeoPixel pixel(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  // Inicializa el NeoPixel
  pixel.begin();

  // Apaga el LED al inicio
  pixel.clear();
  pixel.show();
}

void loop() {
  // Encender en rojo
  pixel.setPixelColor(0, pixel.Color(255, 0, 0));
  pixel.show();
  delay(1000);
  // Encender en verde
  pixel.setPixelColor(0, pixel.Color(0, 255, 0));
  pixel.show();
  delay(1000);
  // Encender en azul
  pixel.setPixelColor(0, pixel.Color(0, 0, 255));
  pixel.show();
  delay(1000);
}
```

Desarrollo de la práctica en el laboratorio

Identificación de los estudiantes que participaron en la práctica

Por favor suministrar los datos de los estudiantes que participaron en el grupo y desarrollaron la práctica, ojo sólo podrá registrar las personas que asistieron presencialmente y que efectivamente aportaron el desarrollo de la práctica:

Fecha de entrega: _____ Profesor: _____

Documento de identidad	Nombre y apellido	Observaciones

Planteamiento general de la práctica

Para el desarrollo de esta práctica, el estudiante deberá estudiar cuidadosamente la teoría presentada en el marco conceptual, prestando especial atención a los diferentes esquemas de comunicación serial abordados: la comunicación **UART** con el computador, la comunicación **I2C** con la pantalla **SSD1306** y la comunicación por un solo pin utilizada para controlar el LED RGB tipo **NeoPixel**. A partir de esta base teórica, deberá transcribir, cargar y probar en el microcontrolador cada uno de los códigos propuestos, verificando su funcionamiento de manera individual. El propósito no es solamente ejecutar los programas, sino comprender cómo se realiza cada ejercicio, qué función cumple cada instrucción importante y cómo se relacionan los conceptos de comunicación, manejo de texto, visualización de datos y control de periféricos dentro de un sistema embebido basado en ESP32-S3.

Ejercicio final de la práctica

Como ejercicio final de la práctica, los estudiantes deberán implementar un sistema embebido basado en el **ESP32-S3**, en el cual se integren los conceptos trabajados previamente sobre comunicación serial, lectura analógica, visualización de información en pantalla OLED y control de un LED RGB tipo NeoPixel.

El montaje físico deberá incluir **un potenciómetro** conectado como divisor de voltaje, de manera que entregue al microcontrolador una señal analógica variable entre **0 V y 3.3 V**. También se deberá conectar un **LED externo** con su respectiva resistencia limitadora de corriente y una pantalla OLED **SSD1306** mediante comunicación **I2C**. Adicionalmente, se utilizará el LED RGB tipo **NeoPixel** integrado en la tarjeta de desarrollo ESP32-S3.

El programa deberá recibir caracteres enviados desde el **Monitor Serie** del entorno Arduino. Cada carácter recibido será interpretado como un comando. Dependiendo del carácter ingresado, el ESP32-S3 deberá ejecutar una acción específica: encender o apagar el LED externo, leer el valor analógico del potenciómetro, mostrar información por la terminal serial, imprimir información en la pantalla OLED o modificar el color del LED RGB NeoPixel.

La siguiente tabla resume los comandos que debe implementar el programa:

Carácter recibido	**Acción solicitada**
1	Encender el LED externo.
0	Apagar el LED externo.
V	Leer el valor analógico del potenciómetro y mostrar el voltaje calculado por el Monitor Serie con dos cifras decimales.
v	Leer el valor analógico del potenciómetro y mostrar el voltaje calculado en la pantalla OLED con dos cifras decimales.
X	Leer el valor analógico del potenciómetro y mostrar el valor bruto del ADC en formato hexadecimal por el Monitor Serie.
x	Leer el valor analógico del potenciómetro y mostrar el valor bruto del ADC en formato hexadecimal en la pantalla OLED.
N	Leer el valor analógico del potenciómetro y mostrar el valor bruto del ADC en formato decimal por el Monitor Serie.
n	Leer el valor analógico del potenciómetro y mostrar el valor bruto del ADC en formato decimal en la pantalla OLED.
R	Colocar el NeoPixel en color rojo e informar el color por OLED y Monitor Serie.
G	Colocar el NeoPixel en color verde e informar el color por OLED y Monitor Serie.
B	Colocar el NeoPixel en color azul e informar el color por OLED y Monitor Serie.
M	Colocar el NeoPixel en color magenta e informar el color por OLED y Monitor Serie.
C	Colocar el NeoPixel en color cian e informar el color por OLED y Monitor Serie.
A	Colocar el NeoPixel en color amarillo e informar el color por OLED y Monitor Serie.
Z	Apagar el NeoPixel e informar el estado por OLED y Monitor Serie.

Cuando se reciba un comando asociado al control del NeoPixel, el programa no solo deberá cambiar físicamente el color del LED RGB, sino también informar el nuevo estado del sistema. Por ejemplo, si se recibe el comando M, el NeoPixel deberá colocarse en color **magenta**, la pantalla OLED deberá mostrar un mensaje como:

```
RGB:

MAGENTA
```

y la terminal serial deberá imprimir un mensaje similar:

```
NeoPixel configurado en color MAGENTA
```

De manera equivalente, si se recibe el comando Z, el NeoPixel deberá apagarse, la pantalla OLED deberá mostrar que el RGB está apagado y la terminal serial deberá reportar este cambio de estado.

Si el sistema recibe cualquier otro carácter diferente a los indicados en la tabla, deberá imprimir por el Monitor Serie un mensaje indicando que el carácter recibido no corresponde a ningún comando válido. Por ejemplo:

```
Comando no valido: q
```

El objetivo de este ejercicio es que el estudiante integre en un solo programa varios elementos básicos de un sistema embebido: recepción de comandos por **UART**, lectura de una señal analógica mediante el **ADC**, presentación de datos con texto formateado, comunicación **I2C** con una pantalla OLED y control de un dispositivo RGB mediante comunicación digital por un solo pin. Además, el estudiante deberá demostrar que comprende la diferencia entre mostrar un dato en formato decimal, hexadecimal y como magnitud física calculada en voltios.

Para la entrega, el estudiante deberá mostrar al profesor el montaje funcionando en el microcontrolador. Deberá probar uno por uno los comandos definidos, verificar que el LED externo responde correctamente, que el valor analógico cambia al mover el potenciómetro, que la pantalla OLED muestra la información solicitada y que el NeoPixel cambia de color según los comandos recibidos.

Evaluación de la práctica

La práctica será calificada de la siguiente forma:

Componente evaluado	**Porcentaje**
Entrega del proyecto funcionando correctamente	50 %
Sustentación individual realizada por el profesor a cada integrante del grupo	50 %

Para la entrega del proyecto funcionando, el grupo deberá demostrar que el montaje físico opera correctamente y que todos los comandos definidos en la práctica producen la acción correspondiente. Se verificará el encendido y apagado del LED externo, la lectura del potenciómetro, la impresión de datos por la terminal serial, la visualización de información en la pantalla OLED y el control del color del NeoPixel integrado.

La sustentación será individual. Esto significa que, aunque el montaje se presente en grupo, cada estudiante deberá estar en capacidad de explicar el funcionamiento del circuito, la lógica del programa,

el uso de los comandos por UART, la lectura del ADC, el envío de información a la pantalla OLED por I2C y el control del NeoPixel. Cada integrante debe comprender el código y no limitarse únicamente a presentar el montaje funcionando.

Conexiones sugeridas y forma de evaluación

Para el montaje físico de esta práctica se utilizará **un solo potenciómetro externo**, conectado como divisor de voltaje. Un extremo del potenciómetro debe conectarse a **3.3 V**, el otro extremo a **GND** y el terminal central al pin analógico seleccionado del ESP32-S3, por ejemplo, el **GPIO 5**. De esta forma, al girar el potenciómetro se obtiene una señal analógica variable entre **0 V y 3.3 V**, que será leída por el convertidor analógico a digital del microcontrolador.

Como conexión sugerida, se puede utilizar la siguiente distribución:

Elemento	**Pin del elemento**	**Conexión sugerida en ESP32-S3**
Potenciómetro externo	Extremo 1	3.3 V
Potenciómetro externo	Extremo 2	GND
Potenciómetro externo	Terminal central	GPIO 5, entrada ADC
LED externo	Ánodo	GPIO 4, mediante resistencia de 220 Ω o 330 Ω
LED externo	Cátodo	GND
Pantalla OLED SSD1306	VCC	3.3 V
Pantalla OLED SSD1306	GND	GND
Pantalla OLED SSD1306	SDA	GPIO 8
Pantalla OLED SSD1306	SCL	GPIO 9
NeoPixel integrado	Señal de datos	GPIO 48, ya conectado internamente en la tarjeta

El LED externo debe conectarse siempre con una **resistencia limitadora de corriente**, preferiblemente de **220 Ω** o **330 Ω**, para evitar exigir una corriente excesiva al pin del microcontrolador. La pantalla OLED **SSD1306** debe conectarse mediante comunicación **I2C**, usando **GPIO 8** como SDA y **GPIO 9** como SCL. Es importante no invertir estas señales, ya que SDA corresponde a la línea de datos y SCL a la línea de reloj. El LED RGB tipo **NeoPixel** ya viene incorporado en la tarjeta de desarrollo y normalmente está conectado internamente al **GPIO 48**, por lo cual no se requiere cablearlo externamente. Sin embargo, se debe tener precaución si se desea usar ese mismo pin para otra función, ya que podría interferirse el funcionamiento del NeoPixel.